

Spectra AI

Security Assessment, Orchestrated.

Technical Documentation

Ritvik Indupuri

1 Project overview

Spectra AI is an AI-assisted security assessment application for authorized environments. It connects a browser dashboard to server-side agent flows and command-line scanners. The workflow gathers reconnaissance where applicable, selects scanners for the chosen assessment profile, converts output into structured findings and recommends next steps.

The application exposes six profiles: web, API, code and infrastructure as code, container images, cloud posture, and AI applications. Four visible agent roles handle reconnaissance, scanning, analysis and coordination. A separate model-backed assistant drafts an editable engagement scope.

Capability	Implemented behavior
Assessment configuration	Profile-specific inputs, optional project ID, active-check consent and editable scope.
Agent orchestration	Sequential Genkit flows; model-selected tools within the scanning stage.
Scanner integrations	Nmap plus 11 registered scanning operations across ten scanner families.
Assessment controls	Preflight authorization, invocation policy checks and input validation in process adapters.
Review and reporting	Agent status, logs, severity counts, detailed findings and plain-text export.
Storage	Browser-local completed sessions; optional Supabase authentication and assessment-request registration.

Technical foundation

The repository specifies Next.js 15.3.3, React 18.3.1 and Genkit 1.41.0. TypeScript and Zod define the request and response contracts. Tailwind CSS, Radix UI components and Recharts support the dashboard. The deployment files package the application and scanners in a Node.js 22 container.

Operating model

The normal workflow runs once from configuration through recommendations. It does not automatically feed the coordinator's next steps into another execution cycle. Findings and severity are generated from scanner output by the analysis model; they are not a deterministic proof that an issue is exploitable.

This document describes the connected application paths and distinguishes them from database structures prepared for later expansion. Sections 2-3 explain the diagrams; Sections 4-8 cover agents, scanners, controls and data; Sections 9-11 cover operation, interfaces and verification.

2 System architecture

The browser invokes Next.js server actions. Genkit mediates model requests, while scanner binaries execute in the application runtime. Supabase is optional and supplies session authentication, project records and assessment-request registration when configured.

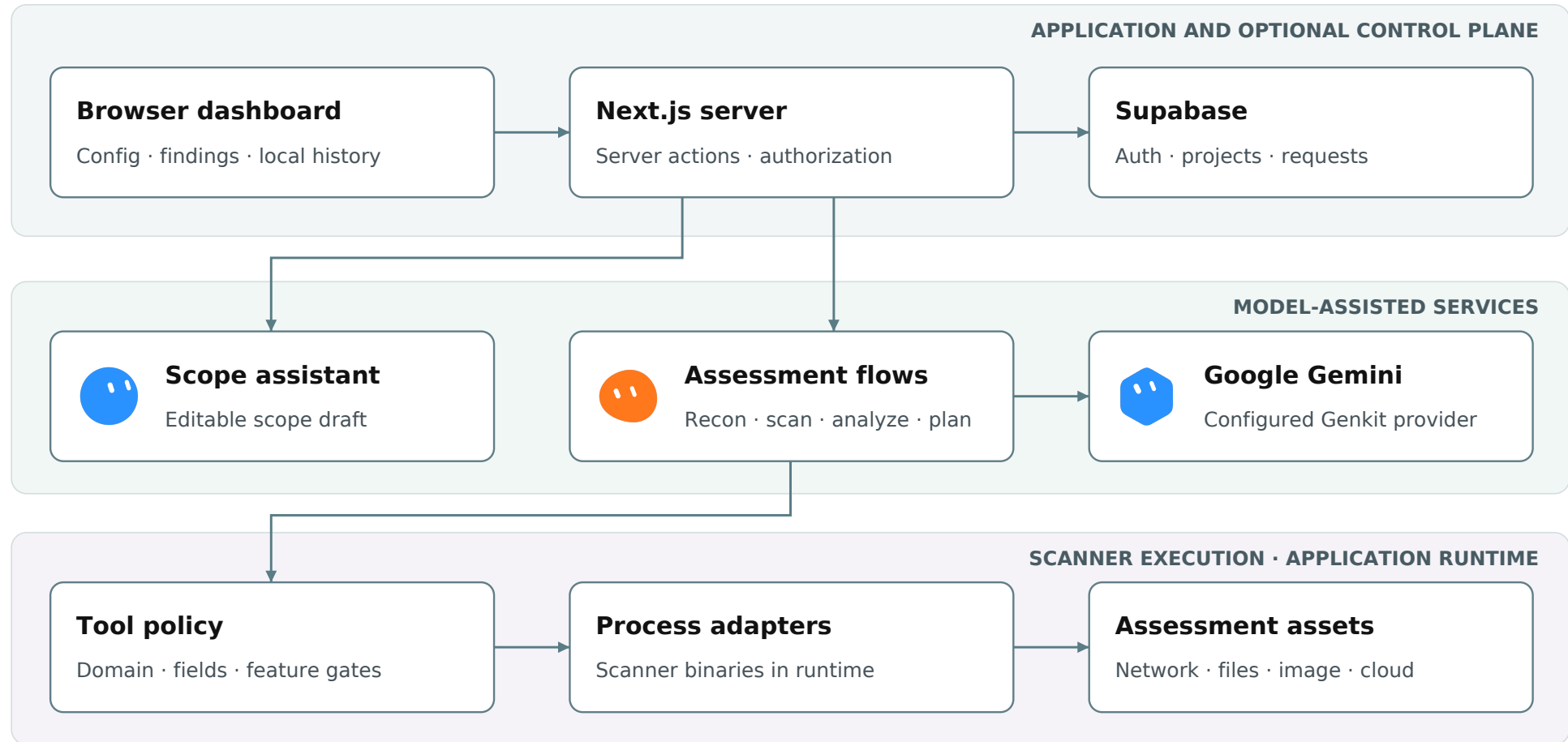


Figure 1. Application components, model services and scanner execution boundary.

The scope assistant also uses the configured Gemini provider. The bottom policy path applies to Scanning Agent tool wrappers; reconnaissance invokes Nmap separately. Results return through the calling flow to the browser. Completed-session history uses localStorage. No independent scan worker or cloud-service topology is implied by this diagram.

3 Agent request lifecycle

The browser controls the sequence and updates agent cards as each stage completes. Web assessments invoke model-assisted reconnaissance; other profiles prepare their supplied inputs and join the scanning stage without network reconnaissance.

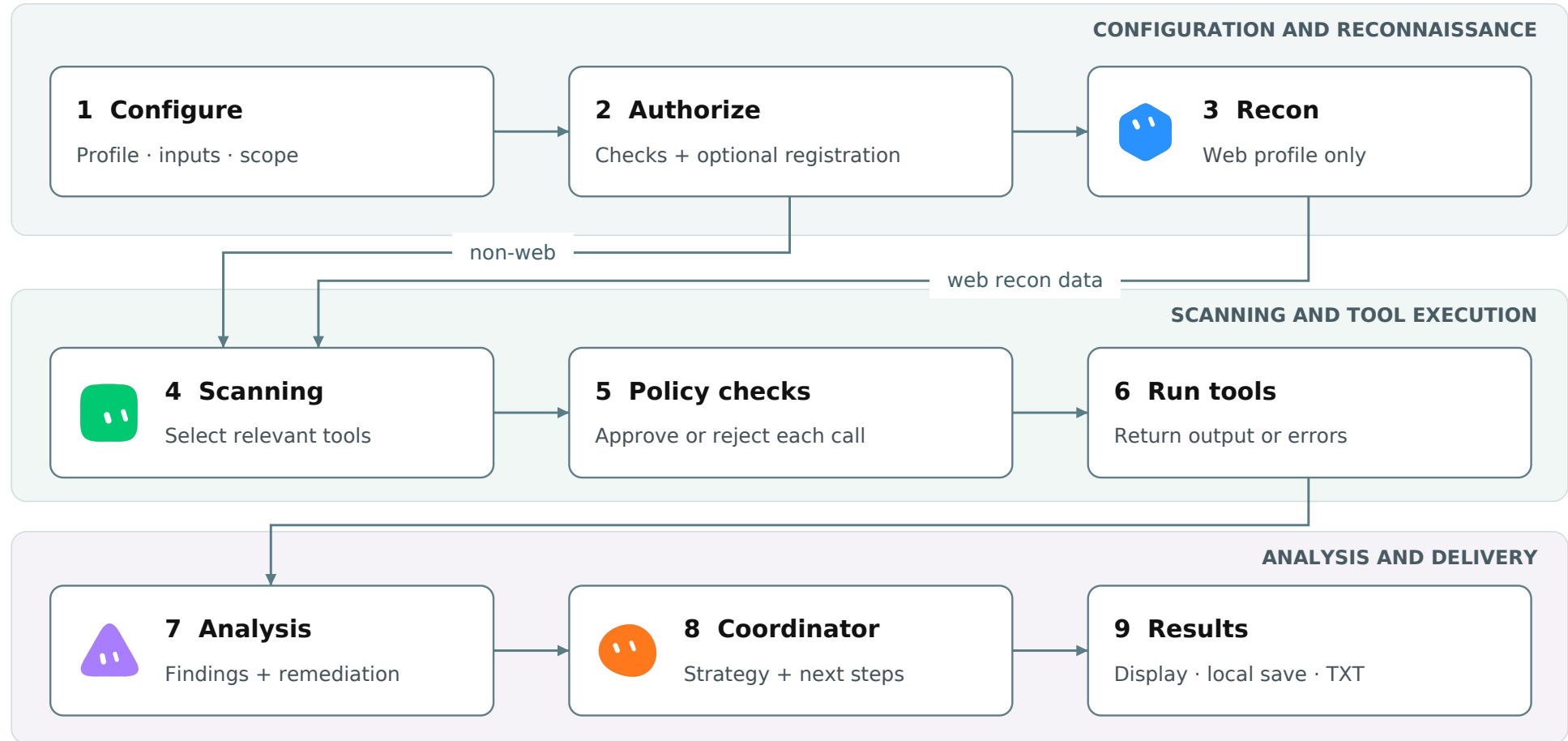


Figure 2. One assessment, including the non-web reconnaissance bypass.

Tool selection can involve multiple model/tool exchanges inside the scanning flow. Analysis receives scanOutput; the coordinator receives the three summaries and scope. The final recommendation is advisory. Scope generation occurs during configuration and is separate from the four visible assessment roles. Agent icons identify model-backed steps only.

4 Agent responsibilities and model contracts

All five flows share the Genkit instance in `src/ai/genkit.ts`, configured with the Google AI plugin and the model identifier `gemini-3.6-flash`. The identifier is the value in source configuration; provider availability and live model behavior were not verified during this documentation review.

Flow	Input and output
Scope assistant	Target; returns a short scope statement. The user can edit the generated text.
Reconnaissance	Target, optional port range and Windows hint; returns <code>reconData</code> , <code>potentialAttackVectors</code> and summary.
Scanning	Profile, supplied assets, <code>reconData</code> and active consent; returns combined <code>scanOutput</code> and summary.
Analysis	<code>scanOutput</code> ; returns report and structured findings.
Coordinator	Recon, scan and analysis summaries plus scope; returns <code>nextSteps</code> , <code>updatedStrategy</code> and rationale.

Reconnaissance and scanning

The reconnaissance prompt exposes Nmap and a PowerShell function. Nmap runs service/version detection. The PowerShell function returns an explicit not-implemented message; it does not establish a remote Windows session. The current Nmap command does not include an operating-system detection flag, despite broader wording in the tool description.

The scanning prompt exposes the registered tools and instructs the model to select relevant checks rather than invoke every tool. Each wrapper calls `enforceToolPolicy` before its scanner adapter. Successful and unsuccessful tool results are supplied to the model, which produces the final `scanOutput` and summary fields.

Analysis and coordination

The analysis schema requires a title, description, severity and recommendation for each finding, with optional references. Allowed severities are Critical, High, Medium, Low and Informational. No CVSS calculation or deterministic cross-tool deduplication function is connected to this step.

The Coordinator card is marked running at the start, but its initial planning messages are assembled by application code. The actual strategic model call occurs after analysis. It considers scope and the three stage summaries; it neither invokes scanner tools nor approves earlier calls.

Logs and evidence

Agent cards combine application-generated progress text with returned model summaries and tool-derived output. The browser awaits each flow result before extracting scan logs. These entries should not be interpreted as a streamed model trace or a complete process event stream.

5 Assessment profiles and tool coverage

Profile	Inputs used	Registered scan operations
Web	Target host; HTTP(S) URL; optional ports	Katana, ZAP Baseline; Nikto and Nuclei with active approval. Nmap runs in reconnaissance.
API	Schema URL or workspace path; optional base URL	Schemathesis; Nuclei when a targetUrl is supplied and active checks are approved.
Code / IaC	Workspace-relative assetPath	Semgrep, Trivy filesystem and Trivy SBOM.
Container	imageRef	Trivy image.
Cloud	cloudProvider	Prowler for aws, azure, gcp, kubernetes or github.
AI application	Target URL, purpose and workspace-relative Promptfoo config	Promptfoo, with AI red-team and active gates enabled.

The registry has 11 operation IDs: nikto, zap-baseline, katana, nuclei, schemathesis, semgrep, trivy-fs, trivy-image, trivy-sbom, prowler and promptfoo. Trivy contributes three operations. Nmap is a separate reconnaissance integration outside this registry.

Selection rules

Tool policy checks the selected domain and required field names. Nikto, Nuclei, Schemathesis and Promptfoo are classified safe-active. Prowler is privileged. ZAP Baseline, Katana, Semgrep and the Trivy operations are classified passive by the application registry; passive web checks can still send traffic to the target.

Trivy SBOM is permitted in the code and container domains when assetPath is supplied. The standard container form exposes the image input, so image scanning is its primary connected operation. Prowler receives the selected provider, while the actual account and permissions come from runtime credentials.

Target-specific configuration

API schemas can be remote HTTP(S) URLs or paths under the scan workspace. Promptfoo uses the provided configuration file as its execution input. The AI target URL and purpose are supplied to the scanning prompt, but the wrapper does not rewrite the configuration's provider or endpoint settings.

A registry entry describes allowed capability; it does not prove that a scanner ran in a particular assessment. The returned tool output, status and analysis must be reviewed to establish what actually executed.

6 Scanner execution and resource limits

security-tools.ts uses execFile with a fixed executable and argument array. It captures stdout and stderr, records elapsed time and returns a structured result. Each invocation has an explicit timeout and an 8 MiB maxBuffer setting; returned streams are also bounded to 8 MiB each.

Operation	Configured command behavior	Timeout
ZAP Baseline	Python baseline script; -I; spider duration -m 2.	5 min
Katana	JSONL output; depth 2; rate limit 5.	5 min
Nuclei	JSONL; rate limit 5; excludes dos, fuzz and intrusive tags.	10 min
Semgrep	scan --config auto --json --metrics off.	10 min
Trivy filesystem	JSON; vulnerability, misconfiguration and secret scanners.	5 min
Trivy image	JSON; vulnerabilities, secrets and image-config misconfiguration checks.	10 min
Trivy SBOM	Filesystem inventory in CycloneDX format.	5 min
Schemathesis	25 examples; one worker; JUnit report under /tmp.	10 min
Prowler	Selected provider; JSON-OCSF output directory under /tmp.	20 min
Promptfoo	redteam run with mounted config and --no-cache.	20 min

Result interpretation

SecurityToolResult contains tool, domain, success, exitCode, durationMs, stdout, stderr and truncated. A zero process exit yields success=true. A rejected process promise yields success=false and preserves available output. Some scanners use nonzero exit codes for findings, so process success must not be equated with a clean assessment.

Prowler writes reports to a generated directory and Schemathesis writes a JUnit file. The adapter returns captured process streams; it does not read those report files back into a separate evidence store. The Scanning Agent's raw-output field is model-returned text rather than an independently persisted process archive.

Nmap and Nikto adapters

The older scanning.ts adapters use shell exec after character filtering. Nmap runs nmap -sV with an optional filtered port list; Nikto runs nikto -h with the filtered target. They return text including an Executing command line or an error message. They do not apply the shared adapter's explicit timeout or 8 MiB limit; Nikto's registry timeout is metadata, not a timeout enforced by this adapter.

7 Authorization and invocation policy

Assessment preflight

The dashboard calls `isAssessmentAuthorized` before `registerAssessment` and the agent flows. In Supabase mode it requires a signed-in user, a visible project and an exact matching `project__targets` value with `verified__at` set and no expired authorization. The candidate value depends on the profile: image reference, code path, cloud provider, or target URL/host.

The web check permits RFC 1918 IPv4 ranges and exactly 127.0.0.1, rejects other IPv4 addresses, and applies a blocked-domain suffix list. IPv6 is accepted by the IP branch without an equivalent private-range test. Hostname checks do not resolve DNS; this is not a DNS-aware network-scope boundary.

Scanning Agent policy

Control	Enforced condition
Domain	The requested tool must list the assessment profile in its registry domains.
Required inputs	The original assessment context must contain the required nonempty fields.
Active checks	Both <code>input.allowActive</code> and <code>ALLOW_ACTIVE_SCANS</code> must be true.
Privileged checks	<code>ALLOW_CLOUD_SCANS</code> must be true for privileged operations.
Tool allowlist	If <code>ALLOWED_SECURITY_TOOLS</code> is nonempty, the operation ID must be present.
Policy context	Calls without the <code>AsyncLocalStorage</code> assessment context are rejected.

`ALLOW_AI_REDTEAM` additionally gates the AI profile in preflight. Registry `enabledByDefault` values do not themselves filter the runtime tool list. The runtime checks one operation per invocation; `maxTools=1` at that point is not a whole-assessment tool-call budget.

Adapter validation and boundary meaning

Web URL adapters require HTTP(S) and reject embedded credentials. Image references use a restricted character pattern. Workspace paths are resolved under `SCAN_WORKSPACE` and checked lexically against that root; this is not a `realpath`-based symlink containment check.

The runtime policy validates the original context and tool ID, not equality between every model-generated tool argument and the verified asset. Reconnaissance is outside that policy wrapper. The dashboard preflight and prompt instructions therefore should not be described as a universal authorization check inside every exported server function.

Project active/privileged flags and per-project tool policies exist in the SQL schema, but `withToolPolicy` reads environment gates and the environment allowlist. It does not load those project policy rows. The documented controls are the checks connected to the present invocation path.

8 State, reports and control-plane data

Completed sessions

The browser stores completed `PentestSession` objects under `pentest-sessions` in `localStorage`. Each includes an ISO-time ID, configuration, four agent states and their outputs/logs, structured findings and completion time. Loading history restores a session in the current browser. Clearing history removes the stored list and resets the dashboard.

`FindingsReport` renders the findings and exports `pentest-report.txt` when the list is nonempty. The export includes severity, title, description, recommendation and references. It is a plain-text findings report, not a PDF export or a full raw-evidence archive.

Supabase schema

Tables	Purpose
organizations; organization_members	Tenant membership and owner/admin/security_engineer/analyst/viewer roles.
projects; project_targets	Project settings and exact-value verified target authorizations.
project_tool_policies	Per-project tool policy definitions in the schema.
assessments	Requested profile, target, scope and lifecycle fields.
scan_jobs	Tool-job metadata, worker identity, attempts and result fields.
findings	Assessment-linked finding records with unique fingerprints.
audit_events	Attributed control-plane actions and metadata.

`request_assessment` checks permitted role, exact verified target value and the count of queued/authorizing/running/cancelling assessments against the project limit. It inserts an assessment and an `assessment.requested` audit event. The returned ID is displayed in a coordinator log.

The current dashboard does not create `scan_jobs`, upload its findings into the findings table or mark the assessment completed. Those schema structures do not establish a connected worker queue. Local browser completion and a database assessment status are separate states in this implementation.

Cancellation and audit scope

The migration defines `request_assessment_cancellation` to update status and append a cancellation-request audit event. No worker cancellation consumer is connected in the inspected application. Direct client writes to `scan_jobs`, findings and `audit_events` are revoked from authenticated and anonymous roles; the audit events implemented here describe control-plane requests, not every tool execution.

9 Installation and deployment

Container setup

The Dockerfile packages Next.js and scanner dependencies together. Compose publishes the application on loopback port 9002 and mounts `./scan-workspace` read-only at `/scan-workspace`. Create the environment file from the supplied example and set `GEMINI_API_KEY` before startup.

```
git clone https://github.com/ritvikindupuri/Spectra-AI---Agentic-Pentesting.git
cd Spectra-AI---Agentic-Pentesting
cp .env.example .env.local
mkdir -p scan-workspace
docker compose up --build -d
```

Open `http://localhost:9002`. Put approved code, schemas and Promptfoo configuration files in `scan-workspace` and refer to them using relative paths. In a container, `127.0.0.1` refers to that container. Compose adds `host.docker.internal` for services on the host; the target must still satisfy the selected authorization path.

Setting	Purpose
<code>GEMINI_API_KEY</code>	Gemini provider credential.
<code>ALLOW_ACTIVE_SCANS</code>	Server gate for safe-active registered operations.
<code>ALLOW_CLOUD_SCANS</code>	Cloud-profile and privileged-tool gate.
<code>ALLOW_AI_REDTTEAM</code>	AI-profile preflight gate.
<code>ALLOWED_SECURITY_TOOLS</code>	Optional comma-separated registry operation IDs.
<code>SCAN_WORKSPACE</code>	Adapter path root; Compose sets <code>/scan-workspace</code> .
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Optional control-plane project URL.
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	Supabase browser/server session key.

Runtime configuration

The image runs as the node user. Compose drops all Linux capabilities, enables no-new-privileges, configures init and provides a 512 MiB `/tmp` tmpfs. The image health check fetches the application every 30 seconds. These are shared-runtime controls, not per-assessment containers or isolation from the model provider.

Tool installation mixes pinned components, including Trivy 0.74.0 and Prowler 5.39.0, with floating images/packages such as Nuclei, Katana, Promptfoo, Semgrep and Schemathesis. Build reproducibility therefore depends on more than the application lockfile.

10 Configuration and operational use

Local development

package.json requires Node.js 22-24 and npm 10 or newer. Local development requires the selected scanner binaries on the server PATH in addition to the JavaScript dependencies. The container recipe supplies those integrations; installing npm packages alone does not install all scanners.

```
npm ci
npm run typecheck
npm run dev
```

Development uses port 9002. Additional scripts provide build, start, lint, genkit:dev and genkit:watch. Next.js is configured for standalone output. Although next.config.ts skips build-time lint and TypeScript failures, the Docker builder separately runs typecheck before build.

Optional company configuration

Apply supabase/migrations/202608180001_control_plane.sql to the intended database. Provision the organization, users, memberships, project and verified project target records through an administrative process. The application includes email/password sign-in; it does not include a complete organization provisioning interface.

With both public Supabase variables configured, middleware checks the session and redirects unauthenticated requests to /login. The assessment form uses the application projects table UUID as company project ID. It is not the Supabase service's dashboard project reference.

NEXT_PUBLIC values are used by client code and need to be available to the Next.js build. Compose supplies .env.local at runtime but does not declare explicit public-variable build arguments. For an authenticated deployment, verify the build environment includes the same Supabase public configuration as the runtime.

Running and reviewing an assessment

Select the profile, supply its required fields, review the generated scope and confirm authorized use. Enable active checks only when needed and ensure the matching server gate is set. Web recon runs independently of the registered safe-active consent gate, so leaving that consent off does not mean no network probe will occur.

Review returned scanner output and failures before treating generated findings as verified issues. For cloud work, configure the runtime credentials for the intended provider and account; the wrapper inherits process.env and does not issue or downgrade credentials. Promptfoo configurations likewise need to identify the authorized endpoint and provider.

Progress and recovery

Agent states are idle, running, completed or error. An exception marks currently running cards as error and ends the client orchestration. A completed session is saved after the final coordinator call. Refreshing or closing the browser is not a reliable cancellation mechanism for a server process; the SQL cancellation request is not wired to process termination.

11 References and verification

The documentation was checked against source at the revision below on the repository's master branch. The diagrams describe connected call paths. Agent summaries, scanner tooltips and roadmap statements were not treated as evidence of additional runtime behavior.

Revision: 7be4171979b2bc6471477654b63c3a0c2f4c3b24

Subject	Primary source
Client orchestration	app/page.tsx
Configuration form	pentest-config.tsx
Model configuration	ai/genkit.ts
Agent flows	recon-analysis.ts; scanning-agent.ts; smart-scan-report.ts
Scope and strategy	generate-scope.ts; dynamic-decision-making.ts
Authorization and policy	auth-server.ts; tool-registry.ts; tool-policy-context.ts
Process execution	scanning.ts; security-tools.ts
Control plane	control-plane.ts; control-plane SQL migration
Authentication and export	middleware.ts; login/actions.ts; findings-report.tsx
Build and deployment	package.json; Dockerfile; compose.yaml; .env.example

Validation scope

Source review covered the dashboard sequence, all five model flows, scanner adapters, policy context, authorization, SQL migration and container configuration. PDF pages and diagrams were rendered and inspected for layout. No assessment traffic, cloud scan, model request or live database operation was performed for this document.

Next steps

Further development can connect durable workers, completion updates, stored findings and cancellation; bind every tool argument to approved assets; and centralize execution limits across reconnaissance and scanning. Reproducible scanner builds and automated contract tests would also strengthen deployment validation. These are follow-on improvements rather than capabilities attributed to the current workflow.